Student Name: Abdulwahab Hassan
Program: Student Record Management System
Date: 2026-10-31

Program Explanation: This is a C++ Student Record Management System which will load the student records (ID, score and grade), manage them and then allow you to retrieve a student's record. It does this by first reading in the student records (ID, score and grade) from a local `students.csv` file into an array of `Student` objects. It then displays the records and asks you to add a new student. It has input validation loops to ensure that the score and grade you enter are in the valid range. Once the new student record has been added it will sort the entire array alphabetically by student ID using a bubble sort. It will then use a binary search to find and display the record for a particular student based on the ID you enter.

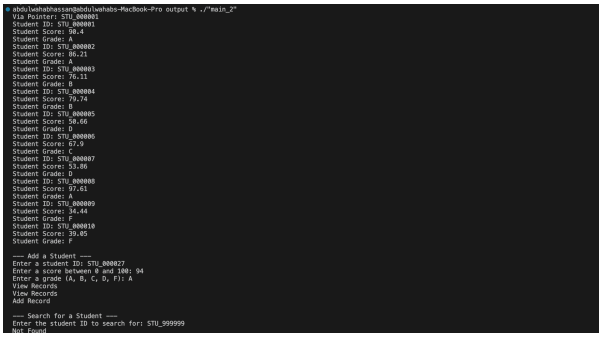
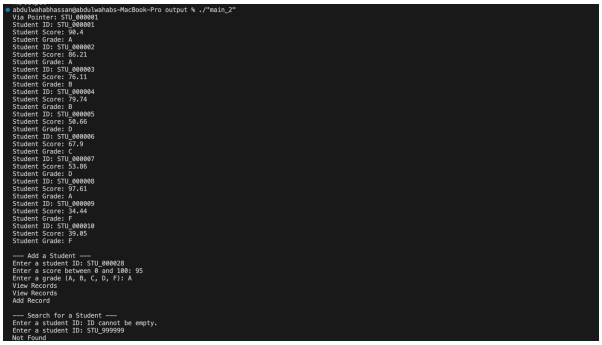
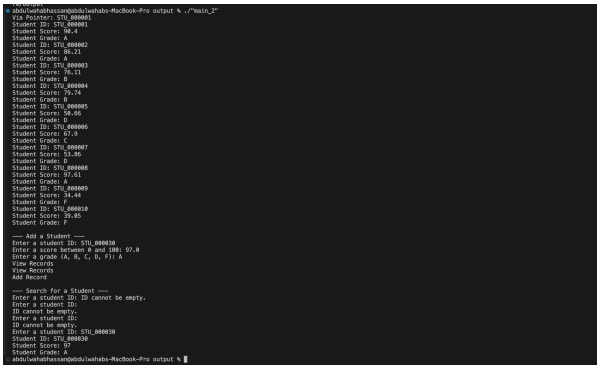
Concepts Used

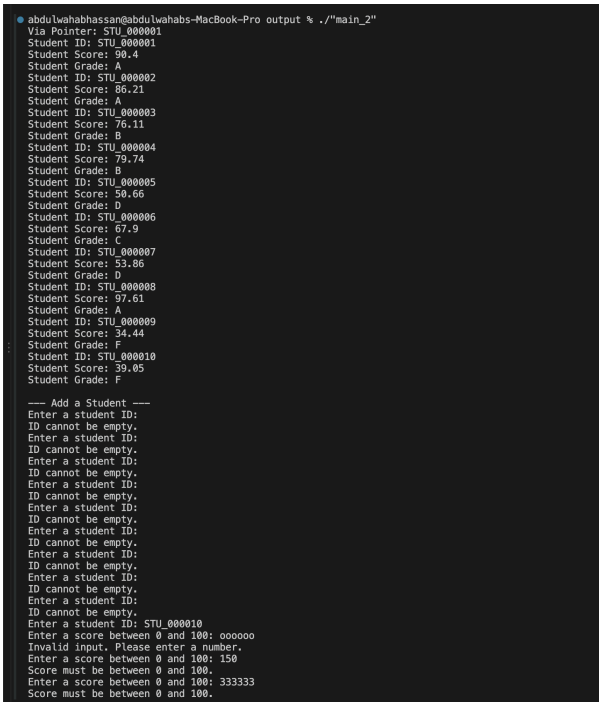
- **Object Oriented Programming (classes and objects) with a Student class which has the** attributes (ID, score, grade) and the appropriate methods (getters, setters, display etc).
- **Inheritance:** creating a class hierarchy with a base `User` class and a derived `Admin` class to illustrate the concept of access privileges.
- **File Input/Output (I/O):** Utilizing `ifstream` and `stringstream` to open, read, and parse comma-separated values from an external file.
- **Arrays:** storing a collection of `Student` objects using a standard fixed size array.
- **Pointers:** Using memory addresses to directly reference and manipulate objects (e.g., `Student* ptr = &student[0];`).
- **Sorting Algorithms:** Implementing a basic bubble sort to organize the array of students alphabetically by ID.
- **Searching Algorithms:** Implementing a binary search function for highly efficient record retrieval on the sorted array.
- **Input Validation:** Using `while` loops alongside `cin.fail()`, `cin.clear()`, and `cin.ignore()` to catch erroneous data types and enforce range rules without crashing the program.
- **Data Types and String:** Multiple C++ data types are used according to the nature of each field. String for student IDs, double for exam scores, char for letter grades and int for counters and array indices.

How to Run

Compile with g++ main_2.cpp -o main_2. The file students.csv must be present in the parent directory of the executable, so the program reads from ../students.csv.

Screenshot of the Program Execution

	Loading ten records from students.csv into an array of student objects, with the first record accessed through a pointer, followed by adding a new student record (STU_000027)
	Searching for a student ID that does not exist (STU_999999). The binary search returns -1 and the program reports “Not Found” rather than failing.
	Adding a new student record (STU_000030) and searching for a student record (STU_000030).

	Image of the program running with a score input test; typing in the wrong input to see how the program reacts to it and as you can see it states and prompts to input the correct input.

Note:
Letter grades must be entered in uppercase.